Conference Paper Title

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Malicious code poisoning in the AI model supply chain has emerged as a critical and rapidly growing threat. Unlike traditional software, model files are opaque binary artifacts routinely downloaded and executed with minimal inspection from platforms such as Hugging Face [1]. Recent attack reports reveal that compromised models have already caused large-scale theft of user credentials and illicit exploitation of LLM API tokens, affecting hundreds of thousands of end users and exposing CI/CD secrets of major service providers [2]–[4]. These incidents demonstrate that a single poisoned model artifact can propagate broadly through the reuse-driven AI ecosystem, amplifying the blast radius of each attack [5].

To defend against these threats, the majority of existing research has focused on static detection methods, which identify potential risks by scanning serialized files for known malicious signatures, decompiling pickle files, or detecting suspicious import behaviors [6]–[8]. These tools reason about code structure rather than executed behavior, so they cannot distinguish a benign model loading remote weights from a malicious payload establishing a reverse shell when both invoke the same networking library. Therefore there is a growing consensus that runtime detection is necessary to effectively mitigate these threats [9], [10]. However, existing dynamic detection methods rely on prior knowledge of attack patterns to instrument specific functions or filter system events, which makes them vulnerable to rapidly evolving adversarial techniques that embed payloads directly into computational graphs using only legitimate framework APIs [11]. This results in high false-positive and false-negative rates.

We identify the key limitation of existing defense strategies as their reliance on prior knowledge of attack patterns. Static detection tools scan serialized files for known malicious signatures, import dangerous function blacklists, or decompile pickle bytecodes to identify suspicious patterns. The problem is even more severe for dynamic methods [9]: instrumentation techniques hard-code which specific functions to hook based on expert heuristics, and system call monitors depend on static whitelists to filter benign events. Because adversarial techniques iterate rapidly any defense contingent on knowing what to look for is inherently vulnerable, suffering from both high false-positive and false-negative rates and complete defenselessness against zero-day exploits that lack established signatures.

To overcome this reliance on prior knowledge, we shift our strategy from hunting known threats to modeling benign behavioral characteristics. Our key observation is that benign AI models exhibit highly regular runtime templates, governed by the framework's internal logic and largely invariant across model architectures and tasks. Normal execution consistently passes through a small set of stable phases—deserialization, tensor materialization, and library loading—while malicious models inevitably deviate from these templates, producing anomalous patterns such as unexpected file access during deserialization or unauthorized network communication during inference.

Building on this observation, we propose MAD, a defense that requires no prior knowledge of specific attack methodologies. MAD transforms dynamic model security protection into a two-stage behavior analysis problem, bridging the semantic gap between low-level system evidence and model intent. In the first stage, an unsupervised model learns the distribution of benign operation patterns from system call traces and efficiently filters out normal activity, requiring no attack signatures or whitelists. Only statistically abnormal operations are escalated to the second stage, where they are associated with high-dimensional Python semantic context collected via lightweight, non-invasive instrumentation. This cross-layer association combines non-bypassable system call evidence with application-layer semantics. Finally, the LLM is used to trace the root cause of underlying operations and judge whether such behaviors conform to the regular execution logic of the model or belong to malicious attacks. Through this two-stage pipeline, MAD achieves both high detection accuracy and low false-positive rates while maintaining practical efficiency.

However, realizing MAD faces three non-trivial technical challenges. First, how to define a meaningful unit of system call behavior that can be learned by a machine learning model. A single system call is excessively fine-grained and massive in volume, while malicious behaviors are generally manifested via consecutive correlated call sequences. MAD addresses this by aggregating system calls into lifecycle-bounded operations, converting raw traces into semantically meaningful operation vectors. Second, how to achieve comprehensive semantic instrumentation across heterogeneous, rapidly evolving AI frameworks without requiring source-code access or imposing prohibitive engineering overhead. MAD leverages an LLM to dynamically generate framework-adaptive monkey-patching scripts, which automatically discover and wrap all public callable functions at runtime. Third, how to reliably align asynchronous Python-level semantic events with kernel-

level system call traces under high concurrency, where naive timestamp matching produces spurious associations. MAD addresses this through a fuzzy spatio-temporal alignment strategy that combines temporal windowing with resource-related spatial hints to robustly associate each abnormal operation with its true application-layer context.

Experiments on 89 real-world and 80 advanced synthetic malicious models demonstrate that MAD achieves 100% precision, 98.70% F1-score, and 0% false positives on benign models using sensitive framework APIs, while introducing only 5.69% average runtime overhead. In summary, our contributions are as follows:

- We propose the first hybrid dynamic detection system for AI model supply chain attacks that combines non-bypassable system-call evidence with Python semantic context, bridging the semantic gap between low-level behaviors and model intent.
- We develop a robust cross-layer association mechanism that aligns abnormal system-level operations with high-level semantic events, enabling LLM-based reasoning to provide accurate and explainable security alerts.
- Extensive experiments on real-world and synthetic malicious models demonstrate that our system achieves high detection accuracy and low false positive rates, outperforming existing static defenses with good practical deployment value.

II. Background

A. AI Model Supply Chain Attack

AI model supply chain. The rapid advancement of machine learning and the proliferation of open-source model hubs have fundamentally transformed how AI models are developed and deployed. Developers increasingly upload their pre-trained models to centralized platforms such as Hugging Face, enabling the broader community to freely download, fine-tune, and integrate these artifacts into downstream applications [12]. This collaborative ecosystem forms the core of the AI model supply chain. According to Hugging Face statistics, the platform hosts over 2.8 million models as of 2026, supporting more than 1,000 languages and dozens of distinct tasks [13], [14]. Foundational models like BERT and GPT have become indispensable tools for developers, each recording over ten million downloads. The widespread sharing of models further demonstrates the critical role of the model supply chain in advancing academic research and industrial technological development [15].

AI model supply chain attack. While this model sharing paradigm greatly facilitates collaborative innovation, it simultaneously introduces severe security vulnerabilities to the AI model supply chain. Recent security reports have uncovered structural vulnerabilities on platforms such as Hugging Face [16]. Consequently, a surge of real-world malicious attacks targeting this supply chain has been reported [17]. As illustrated in Figure 1, the typical workflow of these attacks involves malicious actors uploading compromised models to public hubs; these models are specifically designed to execute hidden payloads when downloaded and deployed by unsuspecting users. These supply chain attacks are not merely theoretical; they have already resulted in the large-scale theft of user credentials and the illicit exploitation of Large Language Model API tokens, causing significant financial and operational damage to the affected organizations [2].

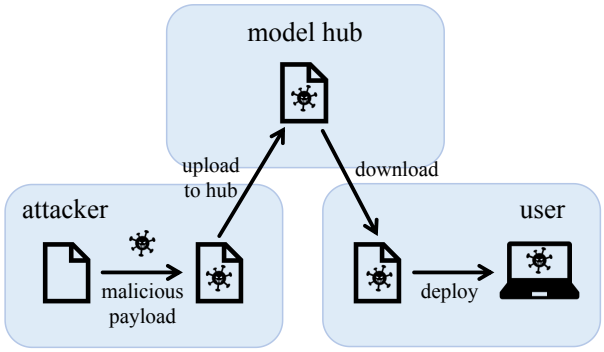

Fig. 1: The workflow of an AI model supply chain attack.

AI model supply chains are highly vulnerable to such attacks mainly because malicious code is implanted stealthily and hard to detect. Unlike traditional software, AI models are often distributed as opaque binary artifacts []. Attackers exploit this by embedding malicious payloads directly into the model structure, using various obfuscation techniques [18]. For instance, malicious code can be intricately disguised as normal model components or hidden within the unreadable binary formats of the model files to successfully evade the static security scanners employed by model hosting platforms [19].

Recent studies have provided comprehensive analyses of these malware poisoning attacks within pre-trained model hubs [20]. Particular attention has been drawn to the exploitation of insecure serialization formats [21], which allow attackers to embed arbitrary execution commands that trigger automatically upon model loading. Since the malicious payload execution is fully integrated into the normal deserialization process, it can easily bypass conventional superficial detection methods and pose a major challenge to security defense [22].

B. Existing Attack Vectors

Current attack methodologies targeting the AI model supply chain can be broadly classified into two categories: insecure serialization attacks that exploit model file formats to embed executable payloads, and framework-native attacks that weaponize legitimate framework APIs to execute malicious operations under the guise of normal tensor computations.

Insecure serialization attacks arise from a fundamental trade-off between architectural flexibility and system security. To accurately preserve complex model architectures alongside their learned weights, frameworks rely on serialization mechanisms that support arbitrary object instantiation [23], effectively blurring the boundary between passive data and executable code [24]. Consequently, when a victim loads a

TABLE I: model formats and their vulnerability to code injection attacks.

Model Format	Primary Frameworks	Vulnerability Level
`pickle`, `marshal`	PyTorch, Scikit-learn	Highly Vulnerable
`joblib`, `dill`	Scikit-learn, Ray, PySpark	Highly Vulnerable
`SavedModel`	TensorFlow	Potentially Vulnerable
`HDF5`	Keras	Potentially Vulnerable
`Checkpoint`	TensorFlow	Secure
`Safetensors`	Hugging Face	Secure
`ONNX`	ONNX	Secure
`JSON`, `MsgPack`	Various	Secure

compromised artifact, the deserialization engine automatically executes the embedded malicious payload prior to any machine learning computations. Recent studies confirm that insecure serialization remains widespread on public model hubs, and that malicious artifacts can be published and propagated through ordinary sharing workflows [25], [26].

The vulnerability level of a model heavily depends on its storage paradigm. Formats designed to preserve both the complex model architecture and the learned weights pose the highest risk. For example, the native Python pickle module and its derivative formats, which are widely adopted by PyTorch and Scikit-learn, support arbitrary object instantiation and are highly vulnerable to code injection attacks. [27], [28]. Similarly, TensorFlow's SavedModel and Keras's HDF5 [29] formats encapsulate rich execution graphs or custom layer definitions, presenting potential attack surfaces when loading untrusted artifacts. In contrast, modern formats restricted to storing only raw numerical weights are inherently secure against direct code injection. Prominent examples include Hugging Face's Safetensors [30], which strictly parses flat tensor data, and the Open Neural Network Exchange(ONNX) format [31], which relies on a rigid schema rather than executable logic. Table I summarizes the security posture of these widely used model serialization formats.

Framework-native attacks weaponize legitimate, built-in APIs of AI frameworks to execute malicious operations without introducing obviously malicious syntax. In these scenarios, malicious logic is compiled directly into the model's computational graph rather than embedded as standalone executable scripts. Frameworks like TensorFlow offer extensive lower-level APIs for distributed training or local debugging that inherently possess capabilities such as file system access or network communication [32]. Attackers hijack these standard operations to perform unauthorized actions—for example, utilizing officially supported debugging APIs to exfiltrate local data to remote servers. The TensorAbuse framework [11] systematically demonstrates this paradigm: attackers can design dangerous operators directly into the inference pipeline, enabling file exfiltration, IP exposure, code injection, and remote shell acquisition, all under the disguise of normal tensor operations.

This technique makes the attack exceptionally stealthy and difficult to mitigate. Traditional static analysis tools typically scan for known malicious signatures or external operating system library imports. Because framework-native attacks exclusively utilize the internal ecosystem of the AI framework, they generate no external dependencies and easily bypass conventional static blacklists. The malicious operations remain dormant during the initial loading phase and are only triggered dynamically as tensors flow through the graph during model inference. Furthermore, This execution dynamic creates a severe semantic gap where security monitors lacking application-layer context cannot easily distinguish between a legitimate data loading operation and a malicious file exfiltration.

C. Existing Defenses and Their Limitations

While extensive research has focused on inference-time threats such as adversarial examples and data poisoning, efforts to secure the AI model supply chain against malware injection remains limited and technically simplistic. Current defensive strategies primarily rely on static analysis and signature-based scanning tools to inspect model artifacts before deployment.

For instance, Fickling [7] was developed as a decompiler, static analyzer, and bytecode rewriter specifically for Python's pickle object serialization. By decompiling the opaque pickle data stream into readable Python code, Fickling attempts to identify embedded malicious payloads. Similarly, the Hugging Face platform employs a custom security scanner [8] which combines the traditional antivirus engine ClamAV [33] with a mechanism to extract and inspect the import tables within pickle files. Furthermore, in tandem with the disclosure of the TensorAbuse attack, a detection methodology based on statically searching for suspicious API calls within computational graphs was proposed.

The foremost limitation of these tools is their heavy reliance on static pattern matching—specifically, detecting the presence of predefined libraries or suspicious function calls. Because they do not analyze the actual dynamic execution of the code, they fail to establish semantic context. A benign model component fetching remote weights and a malicious payload establishing a reverse shell might both utilize the same network-related library. This lack of runtime semantic analysis inevitably leads to high rates of both false positives and false negatives.

On the other hand, because these works generally lack a comprehensive understanding of the evolving attack vectors within the AI model supply chain, their designs are inherently reactive. They are typically engineered to address specific, previously observed vulnerabilities rather than adopting a holistic approach to model behavior. This strategy severely restricts their ability to defend against zero-day exploits or novel attack methodologies that do not conform to known signatures.

III. Threat Model

In this paper, we focus on malicious code poisoning attacks within the AI model supply chain, in which malicious actors exploit the collaborative nature of pre-trained model hubs to distribute compromised artifacts and compromise victim systems.

Attackers. The attackers are malicious AI model providers or actors who have compromised legitimate developer accounts [34]. They have the capability to create and upload malicious datasets and binary model files to public model hubs without providing the original training source code. To embed their payloads, attackers can exploit insecure serialization formats [28] or abuse legitimate, hidden capabilities of framework APIs [11] to bypass static security scanners. They may also employ techniques like exploiting leaked authentication tokens or hijack trusted identities, thereby deceiving the community [35]. Their primary goal is to execute unauthorized actions, such as establishing reverse shells, exfiltrating sensitive data such as user credentials, or deploying ransomware, without attracting attention.

Victim Users. The victims are AI developers and researchers who seek to download and reuse pre-trained models or datasets for their applications []. They generally trust content hosted on well-known model hubs and popular contributors, often prioritizing convenience and efficiency over rigorous security inspections. Victims typically load and execute these models using standard libraries and standard user-level permissions, rather than sandboxed environments. During the standard model loading or inference processes, they unknowingly trigger the embedded malicious code or behaviors [5].

Model Hubs. Model hubs represented by Hugging Face and TensorFlow Hub serve as core platforms for the centralized storage, retrieval, and distribution of AI resources. While they implement certain security measures, such as basic malware scanning, these defenses often lag behind rapidly evolving threats and fail to perform deep semantic-level analysis. The platforms treat models as opaque binary files and typically guide users to download and run them directly without sandboxing. Consequently, they struggle to detect sophisticated attacks that disguise malicious code as normal model components or abuse legitimate framework operations.

IV. Design

In this section, we present the design of our hybrid defense system for detecting malicious behaviors embedded in model artifacts. We first formalize the problem setting and design goals of the system. Then we provide a high-level system overview and detail three core modules that jointly model (i) non-bypassable low-level system-call evidence and (ii) high-dimensional Python semantic context collected via non-invasive instrumentation. Finally, we discuss deployment considerations that enable practical and safe use in real-world analysis environments.

A. System Overview

Input. The system takes a model artifact M (e.g., a checkpoint/weights file, a serialized object, or a model package) and a controlled execution configuration E.

Observations. During execution, we collect two synchronized streams: an application-layer semantic log stream $L = \{(\ell_i, t_i)\}$ from Python instrumentation, and a kernel-visible system-call stream $S = \{(s_j, \tau_j)\}$ from system-call tracing.

Output. The system outputs a structured alert

$$A = \langle \textit{IntentScore},\ \textit{ThreatCategory},\ \textit{Context} \rangle, \qquad (1)$$

where *IntentScore* $\in [0, 1]$ reflects the likelihood of malicious intent, *ThreatCategory* is the predicted attack type, and *Context* is an evidence chain linking semantic context to low-level behaviors, which contains abnormal system-call operations and aligned high-level Python semantic events.

Goals. Our design targets three goals: (G1) *non-bypassability* via system-call evidence; (G2) *low false positives and explainability* via semantic context; and (G3) *efficiency and deployability* via fast prefiltering and lightweight instrumentation.

Module 1: Kernel-level behavior extraction and prefiltering. We aggregate raw system calls into higher-level "operations" and use an Autoencoder to quickly identify abnormal operations. This module turns noisy, fine-grained traces into compact operation vectors and filters out clearly benign behavior, ensuring that only suspicious low-level activities are forwarded for deeper analysis.

Module 2: Application-layer semantic instrumentation. We perform non-invasive, framework-adaptive Python instrumentation to capture high-dimensional semantic context around security-relevant API calls. This module logs which libraries and APIs are invoked, with what summarized arguments and in which execution phase, providing the semantic side of the evidence chain.

Module 3: Cross-layer association and LLM reasoning. We align abnormal low-level operations with nearby semantic contexts, then ask an LLM to judge whether the behavior is explainable under benign model execution, producing an intent score and an evidence chain. This module links what the model code appears to be doing with what the operating system actually observes, and converts the combined evidence into a human-understandable alert.

B. Kernel-level Behavior Extraction and Prefiltering

This module processes raw system-call traces captured during model execution and converts them into a compact set of suspicious "operations" for downstream cross-layer analysis. We base our detection on system-call traces because they operate below the user-space level and cannot be bypassed by any Python-level obfuscation [36]: no matter how a model artifact hides its malicious intent through code mangling or dynamic loading, its actual interactions with the operating system must traverse the kernel and therefore leave a non-bypassable forensic record.

However, directly modeling individual system calls is impractical because they are extremely fine-grained and voluminous. A single model loading process may produce tens of thousands of system calls. Moreover, meaningful malicious behavior invariably manifests as a *sequence* of related calls rather than as isolated events. Accordingly, by aggregating correlated system calls into high-level operational units, this module could reduces trace noise, compresses data volume, and outputs standardized analysis units with complete semantics that facilitate subsequent anomaly detection computations.

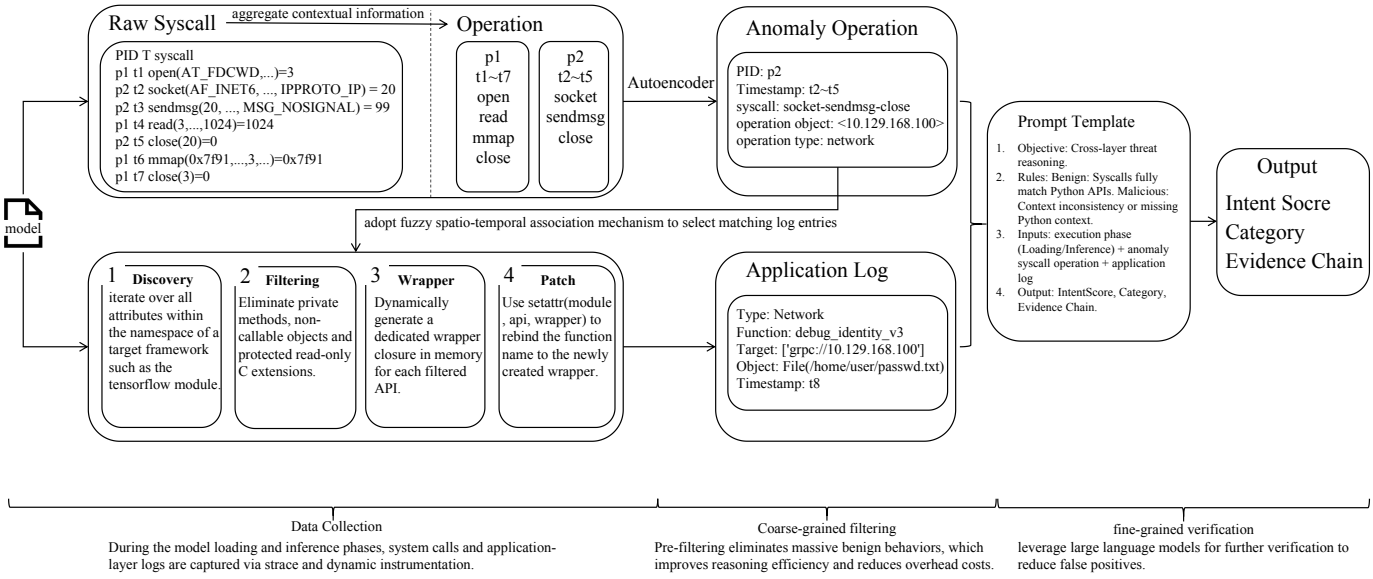

Fig. 2: Overview of the hybrid defense system.

TABLE II: Security-relevant system-call categories used by our filter.

Filter param	Category	Purpose
file	File operations (`read`/`write`)	Detect sensitive data leakage or configuration file tampering
network	Network operations (`socket`)	Detect data exfiltration or remote command execution
cred	Credential operations (`setuid`)	Detect privilege-escalation behavior
desc	Descriptor operations (`dup`)	Assist file-related anomaly detection
signal	Signal handling (`kill`)	Capture malicious process termination attempts
clock	Time-related operations	Assist temporal anomaly analysis (e.g., abnormal delays)

System-call collection and filtering. We employ a selective tracing strategy that focuses exclusively on security-relevant system-call families, discarding high-frequency but semantically uninformative calls to keep the trace volume manageable. During controlled model execution, we use `strace` to continuously trace system calls of the target process. To reduce noise and overhead, we immediately discard calls such as scheduler hints and memory-mapping operations that carry no security signal [37]. The selection concentrates on system-call categories that can indicate malicious intent. Table II summarizes the security-relevant syscall families we monitor and their roles in our detection pipeline. For example, file operations can reveal attempts to access sensitive data or modify configurations, while network operations may signal data exfiltration or remote command execution.

To accurately model the execution semantics of a program, it is crucial to recognize that system calls inherently possess varying degrees of contextual dependency. Applying a uniform extraction strategy to all system calls would be ineffective: some operations are semantically self-contained and immediately indicative of high-risk behavior, whereas others are granular, stateful operations that remain ambiguous unless analyzed as a continuous sequence. Motivated by this fundamental difference, we propose a hybrid embedding scheme that categorizes the traced system calls into two distinct streams for targeted handling: (1) *Directly malicious calls* and (2) *Stateful calls*. By processing these two streams individually, we ultimately synthesize them into a unified feature space for downstream analysis.

Direct call embedding. The first category includes standalone, high-risk operations (e.g., `execve`, `chmod`) that do not require lifecycle context. We directly encode each of these calls into a fixed-dimension operation vector $\mathbf{v}_{dir} \in \mathbb{R}^d$. Specifically, we utilize a pre-trained BERT-based model to extract the rich semantic meaning from the call's underlying logic and literal arguments. This high-dimensional representation is subsequently projected down to the target dimension d, ensuring that semantically critical, self-contained threats are instantly captured without the need for sequence aggregation.

Stateful aggregation. Let $S = \{(s_k, \tau_k)\}_{k=1}^{N}$ denote the traced system-call stream, where s_k is the k-th syscall and τ_k its timestamp. For stateful operations, we maintain per-resource state keyed by $r = \langle \mathit{PID}, \mathit{FD} \rangle$, where *FD* denotes a file or socket descriptor. For each key r, we collect the indices of calls that operate on this resource within one lifecycle segment,

$$I_r = \{\, k \mid (s_k, \tau_k) \in S,\ s_k \text{ uses resource } r,\ \tau^r_{start} \le \tau_k \le \tau^r_{end} \,\}, \tag{2}$$

where τ^r_{start} and τ^r_{end} are the timestamps of the resource-acquisition and resource-release calls (e.g., `open` / `close` or `socket` / `shutdown`). The lifecycle segment for resource r is then the ordered subsequence $S_r = \{(s_k, \tau_k)\}_{k \in I_r}$.

We aggregate this segment into a fixed-length operation vector by applying a feature-extraction function ϕ over all calls

in the segment:

$$\mathbf{v}_{sys}(r) = \phi(S_r) \in \mathbb{R}^d, \tag{3}$$

where ϕ operates by extracting the common resource identifier $r = \langle PID, FD \rangle$ shared across the segment S_r, and concatenating it with the sequence of distinct, call-specific parameters (e.g., flags, buffer sizes, or return values) derived from each individual system call $s_k \in S_r$. This concatenated feature sequence is subsequently passed through an embedding layer to project it into the continuous $\mathbb{R}^d$ space. Intuitively, each $\mathbf{v}_{sys}(r)$ captures a dense, contextualized representation of the exact data flow and state transitions associated with a specific file or connection during its lifetime, effectively smoothing out scheduling artifacts and interleaving from other threads.

The final feature matrix combines $\mathbf{v}_{dir}$ and $\mathbf{v}_{sys}$, enabling the system to capture both direct and aggregated syscall behaviors. This hybrid representation ensures that directly malicious actions are immediately flagged, while stateful operations are analyzed in context to detect more subtle threats.

(SF: also formalize this part. I think this part can be merged with the previous one.)

Autoencoder-based anomaly detection. We adopt unsupervised anomaly detection because benign model loading and inference produce a stable, learnable distribution of system-call operations, whereas malicious behaviors are inherently rare and diverse, making supervised classification impractical without labeled attack samples that cover the full threat space. Among unsupervised methods, we choose an Autoencoder for its conceptual simplicity and proven effectiveness in reconstruction-based anomaly detection: it learns to faithfully reconstruct benign operation vectors during training and naturally flags any vector it cannot faithfully reconstruct as anomalous.

We train an Autoencoder on operation vectors collected from clean model loading and inference runs. Given an operation vector $\mathbf{v}$, the Autoencoder computes a reconstruction error

$$E = \|\mathbf{v} - Dec(Enc(\mathbf{v}))\|_2^2, \tag{4}$$

where *Enc* and *Dec* denote the encoder and decoder networks, respectively. Operations with reconstruction error E below a threshold θ are treated as consistent with normal model behavior and are discarded. If $E > \theta$, we mark the corresponding operation as abnormal, retain its full underlying system-call segment S' and metadata $\langle PID, TID, t_{start}, t_{end} \rangle$, and promote it to Module 3 for semantic association and intent reasoning.

By performing this unsupervised prefiltering at the operation level, the system keeps the expensive cross-layer association and LLM reasoning off the critical path for the vast majority of benign activity, while still surfacing rare, suspicious low-level behaviors that deviate from the learned distribution of normal AI model execution.

C. Non-invasive Python Semantic Instrumentation

The goal of this module is to collect high-dimensional Python semantic context that explains why a model issues certain low-level operations. The resulting semantic log stream is later correlated with system calls to bridge the semantic gap between model-level intent and kernel-level behavior.

Achieving comprehensive semantic instrumentation is non-trivial.The core challenge is that heterogeneous frameworks expose frequently changing internal abstractions and public APIs, making manual instrumentation both error-prone and labor-intensive. Modifying framework source code or maintaining exhaustive API checklists incurs prohibitive engineering overhead, especially as new frameworks and versions emerge. These challenges motivate our automated, LLM-assisted instrumentation approach that adapts to any framework without hand-engineering.

LLM-assisted Instrumentation. To achieve robust and adaptive monitoring across rapidly evolving AI frameworks, we propose an automated, dynamic API discovery and patching mechanism that eliminates the need for statically hardcoded function lists. Our approach instead leverages Large Language Models to synthesize dynamic probe generation scripts that discover and wrap security-relevant APIs at runtime. The LLM is guided to generate code that iterates over the target module's namespace at process startup, systematically identifying all public callable attributes while bypassing private or internal methods. Rather than manually specifying which APIs to hook, the methodology employs runtime monkey patching to inject logging wrappers around every discovered callable, ensuring comprehensive coverage without per-framework hand-engineering.(SF: this part need more details. what's your prompt? how do you select the API? how you automate the instrumentation?)

The core of our approach lies in a rigorously structured prompt template that constrains the LLM to synthesize comprehensive and safe semantic wrappers. Specifically, the prompt clarifies the security objective of detecting anomalous behaviors during model loading and execution, and imposes rigorous technical constraints to guarantee operational stability of the system. The LLM is instructed to construct wrappers that capture highly detailed execution contexts before and after the original function invocation. This context includes timestamps, operation types, argument values, return types, and the complete call stack retrieved via the `traceback` module. To facilitate automated downstream analysis, the prompt mandates that all intercepted semantic events ℓ_i must be serialized into a standardized JSON format.

The instrumentation is designed to be adaptive: when the underlying framework version changes, we only need to regenerate the wrappers for the new API surface, without changing the core detection pipeline. This enables broad coverage across diverse model frameworks while keeping manual engineering effort low.

The core workflow of the instrumentation is formalized in Algorithm 1. During the `ApplyDynamicPatches` procedure, the script systematically iterates through the target module $\mathcal{M}$. After isolating the public, callable APIs, it dynamically constructs a `Wrapper` closure based on the LLM's specifications. It then employs runtime monkey patching (`setattr`)

Algorithm 1: Automated Dynamic Instrumentation Logic

Input: Target library module $\mathcal{M}$ (e.g., `torch`, `tensorflow`)
Output: Instrumented module capable of context logging

Procedure `ApplyDynamicPatches(`$\mathcal{M}$`)`**:**
  foreach *attribute name* $attr\_name$ **in** dir($\mathcal{M}$) **do**
    if $attr\_name$ *starts with "_"* **then**
      continue
    $target\_func \leftarrow$ getattr($\mathcal{M}, attr\_name$);
    if not *is_callable*($target\_func$) **then**
      continue
    $wrapper \leftarrow$ `Wrapper(`$target\_func, attr\_name$`)`;
    try;
      setattr($\mathcal{M}, attr\_name, wrapper$);
    catch *ReadOnlyAttributeError* ;
      continue ;
Procedure `Wrapper(`$func, name$`)`**:**
  return a closure that captures the full `traceback`, logs to JSON, executes $func$, and delegates return value;

to replace the original function with our intercepted version, incorporating exception handling to safely bypass read-only or immutable attributes.

Data Collection Once the dynamic patches are successfully applied at startup, any invocation of the targeted APIs by the user's code or framework internals will automatically trigger the injected closures. These wrappers execute the semantic extraction logic and serialize the intercepted context into a standardized format. Consequently, each monitored API call is recorded as a structured semantic event:

$$\ell_i = \langle \textit{FuncName},\ \textit{ArgsSummary} \rangle. \tag{5}$$

where *FuncName* typically corresponds to security-relevant framework or standard-library APIs. *ArgsSummary* aggregates only lightweight, non-sensitive information such as tensor shapes and dtypes, configuration flags, truncated or hashed file paths, and bounded-length string summaries. This design ensures that the semantic log stream L captures rich contextual information about the model's execution while controlling logging overhead.

Safety constraints. To reduce the risk of breaking runtime behavior, the instrumentation follows a conservative policy. First, we only wrap callable functions and avoid replacing class definitions, built-in data types, or objects backed by low-level C/C++ bindings. Second, wrappers are kept as thin as possible: they perform best-effort argument summarization and then immediately invoke the original function, without altering return values or side effects. Third, the parameter serialization adopts defensive strategies to control logging overhead by recording only shapes instead of raw tensor contents, truncating long strings, and other similar methods.

Moreover, to preserve the original control and data flow of the ML framework, the generated instrumentation script enforces graceful exception handling throughout the patching process. Modern AI frameworks possess highly complex internal architectures that heavily utilize read-only properties, immutable C-extension bindings, and dynamically generated descriptors. A naive monkey-patching approach that attempts to overwrite these restricted components would inevitably trigger fatal runtime errors, crashing the process during model initialization. To mitigate this risk, our framework's prompt design constrains the LLM to wrap both the dependency injection phase and the runtime execution phase of the generated wrappers within protective `try-except` blocks, ensuring that any patch failure degrades without disrupting the target process.

D. Cross-layer Spatio-temporal Association

The third module receives abnormal operations reported by Module 1 and associates them with high-dimensional Python semantic context from Module 2. Its primary goal is to determine whether each abnormal low-level operation can be reasonably explained as benign model behavior. For suspicious operations that cannot be justified as benign, we further construct a complete cross-layer behavior profile to locate the model code triggering such behaviors and analyze their potential maliciousness.

System calls and Python semantics are inherently asynchronous due to operation system scheduling and concurrent I/O. As a result, naive one-to-one timestamp matching between logs S and L is unreliable in high-concurrency workloads. We therefore adopt a fuzzy spatio-temporal association strategy.

For each abnormal operation, Module 1 provides the underlying system-call segment S' and its metadata $\langle \textit{PID}, t_{start}, t_{end} \rangle$. Given this metadata and the semantic log stream $L = \{(\ell_i, t_i)\}$, Module 3 performs filtering following the spatial-temporal matching rules below.

Temporal windowing. We then apply a slack temporal window around the abnormal operation. For each candidate abnormal operation, we define its active time span $[t_{start}, t_{end}]$ based on the first and last system call in S', and search semantic events whose timestamps t_i fall into an expanded window $[t_{start} - \Delta t, t_{end} + \Delta t]$. The slack parameter Δt tolerates minor scheduling jitter while still localizing semantic context to the period when the resource was actually in use.

Spatial hints. Temporal proximity alone is insufficient under heavy concurrency. We therefore further filter candidates using resource-related hints derived from both layers, such as whether the operation is file- or network-related, file paths or prefixes, remote destinations, and other descriptor metadata when available. Combining temporal and spatial constraints yields a fuzzy but robust alignment that significantly reduces spurious associations when many model components execute in parallel.

Cross-layer behavior profile. After spatio-temporal filtering, Module 3 bundles the abnormal operation vector, its raw system-call summary, and the aligned semantic events into a structured cross-layer behavior profile. This profile serves as the input to the LLM-based intent reasoning stage: it contains (i) what the OS observed at the system-call level, (ii) which high-level Python APIs and call sites were active around that time and on the same resource, and (iii) basic execution-phase information. Together, these elements provide sufficient context to decide whether the abnormal low-level behavior is consistent with benign model execution or indicative of malicious intent.

E. LLM-based Intent Reasoning and Evidence Chain Generation

Determining whether abnormal low-level operations reflect benign model behavior or malicious intent is inherently challenging: the same system-call pattern can arise from legitimate model loading or from an attacker's attempt to exfiltrate data, and without semantic context the two are indistinguishable at the kernel level. To resolve this ambiguity, we feed each abnormal operation together with its aligned semantic contexts into an LLM, which judges whether the behavior is consistent with benign model execution.

Prompt construction. We translate the aligned evidence into a structured natural-language prompt that includes: (i) a concise summary of the abnormal operation including operation type, key statistics, and risk-relevant parameters, (ii) the aligned semantic call contexts including library/API, call stack summary, and argument summaries, and (iii) the execution phase categorized into the model loading stage and inference stage.

Our maliciousness judgment follows a principled cross-layer reasoning rule: a behavior is classified as benign only when its low-level system-call activity can be plausibly explained by the aligned Python-level semantic context. Concretely, if the LLM identifies that the observed file reads, network connections, or process creations are consistent with the framework APIs and execution phase reported in the semantic log, the operation is deemed benign and discarded. Conversely, when inconsistencies exist between the Python-layer semantic logs and the corresponding system-call activities, such as a network connection to an external host with no matching high-level API invocation, or when no matching Python semantic context can be retrieved at all, the behavior is marked as malicious and escalated.

Decision and explanation. The LLM outputs *IntentScore*, a *ThreatCategory* label, and a rationale. Importantly, we persist the subset of semantic events and system-call summaries used in the reasoning as the *evidence chain Context* for auditing.

Fail-safe behavior. If high-risk system-call patterns appear without any plausible semantic explanation (e.g., sensitive network connection with no corresponding high-level network API context), the system raises a high-severity alert and can optionally block execution in the controlled environment.

F. Deployment Considerations

Pre-deployment fuzzing workflow. MAD is designed to operate in a pre-deployment model vetting pipeline rather than in production inference serving. During this vetting phase, a model artifact is executed in a controlled sandbox environment while MAD monitors its behavior. To trigger potentially hidden malicious logic, we employ fuzzing-based testing []: the model is exercised through a diverse set of operations that simulate realistic business workloads, including loading with varied input shapes, invoking inference across different batch sizes, and exercising serialization and checkpointing paths. This fuzzing strategy increases the likelihood of activating dormant malicious payloads that may only execute under specific runtime conditions, providing comprehensive coverage beyond simple model loading.

Runtime overhead and deployability. The primary concern for any runtime defense is its performance impact on the testing workflow. As detailed in our evaluation (Section V), MAD introduces an average execution time overhead of only 5.69% across diverse model architectures, with the overhead amortized over prolonged inference sessions. Within a pre-deployment fuzzing pipeline, this level of overhead is entirely acceptable: the vetting phase is an offline, one-time process conducted before models are released to production, and the marginal additional latency does not affect end-user serving performance. Furthermore, MAD's LLM-based reasoning is only invoked on operations flagged as abnormal by the Autoencoder prefilter, so its token cost scales with the number of truly suspicious events rather than with total runtime, further bounding the operational expense.

V. EVALUATION

We evaluate the effectiveness of MAD in detecting malicious behavior embedded in model artifacts. Our study focuses on three aspects: (i) detection performance compared to state-of-the-art defenses, including robustness to camouflage attacks such as TensorAbuse; (ii) the impact of cross-layer modeling and its components on accuracy and explainability; and (iii) runtime overhead and deployability in realistic analysis environments. We organize the evaluation around the following research questions:

- **RQ1:** What is the performance of MAD in detecting malicious models, and does it show significant advantages over the baselines?
- **RQ2:** What is the runtime overhead of MAD, and is it practical to deploy in real-world model workflows?
- **RQ3:** How does each part of MAD contribute to the detection performance?

A. Experiment Setup

In this section, we describe the datasets, baselines, and implementation details used in our evaluation.

Datasets. We evaluate MAD on two distinct datasets to comprehensively assess its detection capabilities against both known, rudimentary vulnerabilities and complex, advanced supply-chain attacks:

- *MalHug Dataset.* The first dataset consists of 89 models identified as anomalous by the MalHug detection tool from public model repositories. After manual source code auditing and sandbox runtime verification, we label 78 of them as truly malicious models. These samples mainly abuse Pickle deserialization vulnerabilities to implant malicious payloads such as reverse shells. The remaining 11 are benign proof-of-concept samples that only verify the exploitability of insecure deserialization flaws without executing any genuinely destructive operations. While the malicious models represent real-world threats, their attack methodologies rely on well-known deserialization flaws and are generally straightforward to detect using existing static or signature-based scanning tools.
- *Advanced TensorAbuse Synthetic Dataset.* To address the limitations of the MalHug Dataset and evaluate our system against highly stealthy, framework-native attacks, we constructed a second dataset using the TensorAbuse framework. This corpus comprises two complementary subsets. The **malicious subset** contains 80 models generated by embedding malicious payloads directly into the computational graphs of 10 diverse foundation models spanning various domains. Specifically, we designed four categories of advanced attacks: File Exfiltration, IP Exposure, Malicious Code Insertion, and Remote Shell Acquisition. For each category, we implemented two distinct attack methodologies utilizing TensorAbuse primitives, and embedded each of the 8 unique attack variants into the 10 foundation models. The **Benign Subset** consists of 40 models. These samples invoke the same high-risk native TensorFlow APIs regarded as attack entry points in TensorAbuse, yet they are only adopted for legitimate usage scenarios, such as loading local dataset configurations, outputting debug tags, and standard tensor persistence. This subset is specially constructed to precisely quantify the false-positive metric of the proposed scheme.

For all datasets, we treat the model artifact as the unit under test. Labels are assigned by manual inspection of the payload code and by verifying whether a model performs any malicious operation in a sandboxed environment.

Baselines. We select four malicious model detection methods as baselines. These include three widely adopted third-party detection tools—JFrog [38], Protect AI [6], and ClamAV [33]—which represent the state-of-the-art security scanners currently integrated into the Hugging Face platform, alongside the specialized detection tool provided by the TensorAbuse [11] framework.

Crucially, for JFrog, Protect AI, and ClamAV, we directly uploaded our constructed model artifacts to the Hugging Face platform. This methodology triggers the platform's live, integrated security scanning pipelines in real time. By doing so, we capture the authentic detection efficacy and assess the actual defensive posture within a real-world AI model supply chain. For the TensorAbuse detection tool, which focuses on statically identifying suspicious API calls embedded within computational graphs, we utilize the open-source implementation provided by the authors. Since these baseline methods primarily rely on static analysis and signature matching without requiring dynamic execution traces or cross-layer alignment, we evaluate their performance directly against the raw model artifacts under these authentic, platform-native conditions to ensure a rigorous and realistic comparison.

Implementation Details. MAD is implemented in Python. The kernel-level module uses `strace` to collect system calls from the target process and its children. We aggregate raw system-call streams into per-resource operation vectors $\mathbf{v}_{dir}$ and $\mathbf{v}_{sys}$ as described in Section IV, and train an Autoencoder on operation vectors extracted from a corpus of trusted benign models. The reconstruction error threshold θ is chosen on a validation set to balance detection rate and false positives.

For Python semantic instrumentation, we apply non-invasive monkey patching to framework and standard-library APIs that can indirectly perform security- relevant operations (e.g., file I/O, network, subprocess). Instrumentation records events of the form $\ell_i = \langle PID, TID, FuncName, ArgsSummary \rangle$ with bounded argument summaries. Cross-layer association then aligns abnormal operations with nearby semantic events using the spatio-temporal strategy in Section IV, and constructs behavior profiles that are fed to an LLM for final intent reasoning. We use a production-grade LLM with temperature set to zero for deterministic decisions.

All experiments are conducted on a workstation equipped with dual Intel Xeon Silver 4210R CPUs, 128 GB RAM, and two NVIDIA GeForce RTX 3090 GPUs.

B. RQ1: Detection Performance

To answer this research question, we evaluate the detection performance of MAD and the baseline methods across the MalHug Dataset and the Advanced TensorAbuse Synthetic Dataset. This comprehensive evaluation demonstrates MAD's effectiveness in identifying various malicious activities, including known deserialization exploits and sophisticated attacks implemented by leveraging native framework capabilities.

TABLE III: Comprehensive detection performance of MAD and baselines across both datasets. MalHug Dataset metrics are computed on 78 malicious and 11 benign models; Advanced TensorAbuse Synthetic Dataset metrics are computed on 80 advanced malicious models with false positives evaluated on 40 benign TensorFlow-API models.

Method	MalHug Dataset			Advanced TensorAbuse Synthetic Dataset		
	Precision	Recall	F1	Precision	Recall	F1
JFrog	0.8953	0.9872	0.9390	0.8889	0.3750	0.5275
Protect AI	0.8966	**1.0000**	0.9455	0.7273	0.5000	0.5928
ClamAV	0.8904	0.8333	0.8609	–	0.0000	–
TensorAbuse	–	–	–	0.6667	**1.0000**	0.8000
MAD	**1.0000**	0.9744	**0.9870**	**1.0000**	**1.0000**	**1.0000**

Table III summarizes the detection performance across both datasets. On the MalHug Dataset, traditional static scanners achieve moderate to high F1 scores by matching known deserialization signatures. MAD achieves a marginal performance improvement over these tools via accurate intent verification.

However, on the Advanced TensorAbuse Synthetic Dataset, the performance gap expands dramatically. Static tools suffer a sharp drop in detection performance. While TensorAbuse achieves a perfect recall, its precision is only 0.6667, as it fails to distinguish legitimate and malicious calls to TensorFlow APIs. Only MAD maintains consistently high performance on both datasets, achieving an F1 of 0.9870 on the MalHug Dataset and a perfect 1.0000 on the Advanced TensorAbuse Synthetic Dataset. We next analyze each dataset in detail.

Performance on the MalHug Dataset. The MalHug Dataset comprises 89 models, consisting of 78 genuinely malicious models and 11 benign proof-of-concept models. The attacks in this dataset primarily rely on inserting malicious payloads through insecure Pickle deserialization vulnerabilities. Because this attack methodology is well-known and extensively studied, traditional static detection tools such as JFrog, Protect AI, and ClamAV achieve relatively high recall rates by relying on established vulnerability signatures. However, these baseline methods exhibit a critical limitation: they fail to distinguish between actual weaponized attacks and the 11 benign proof-of-concept models. These 11 models merely exploit the inherent mechanics of insecure serialization formats to execute normal operations, demonstrating exploitability without performing genuinely harmful actions like data exfiltration or unauthorized network access. By simply classifying any model containing a vulnerable signature as malicious, these traditional methods generate substantial false positives.

In contrast, MAD successfully detects 76 out of the 78 genuinely malicious models while accurately categorizing all 11 proof-of-concept models as non-malicious. Because MAD relies on a two-stage screening process that evaluates the actual intent of dynamic behavior rather than merely matching static signatures, it demonstrates robust resilience against false positives. Notably, the TensorAbuse detector fails to identify any of these models. Since it is built upon a static graph-based solution, it cannot detect deserialization exploits triggered before computational graph initialization or outside the scope of the computational graph.

TABLE IV: Number of detected malicious models by MAD and baselines on the Advanced TensorAbuse Synthetic Dataset, categorized by attack type (20 models per type, 80 models total).

Method	Attack Type (Total models)				Total Detected
	File Exfil.	IP Exposure	Code Insertion	Remote Shell	
JFrog	20 / 20	0 / 20	0 / 20	10 / 20	30 / 80
Protect AI	20 / 20	0 / 20	0 / 20	20 / 20	40 / 80
ClamAV	0 / 20	0 / 20	0 / 20	0 / 20	0 / 80
TensorAbuse	**20** / 20	**20** / 20	**20** / 20	**20** / 20	**80** / 80
MAD	**20** / 20	**20** / 20	**20** / 20	**20** / 20	**80** / 80

Performance on the Advanced TensorAbuse Synthetic Dataset. We further evaluate MAD on our rigorously constructed dataset of 80 advanced malicious models, where attacks are deeply embedded within the models' computational graphs. Table IV provides a detailed breakdown of the detection results across four distinct attack categories: File Leakage, IP Exposure, Malicious Code Insertion, and Remote Shell Acquisition. Both MAD and the TensorAbuse detection tool successfully detect 100% of the malicious models. The TensorAbuse tool achieves perfect detection here because these specific attacks were generated using its own primitive methodologies, making them highly visible to its static API-checking mechanism. However, as demonstrated in Table III, its static approach fails entirely against attacks outside its predefined scope.

Conversely, MAD achieves this 100% detection rate without relying on static signatures or predefined API lists. By statistically profiling the benign baseline behaviors of models and dynamically intercepting the anomalous system calls generated by malicious payloads, MAD accurately identifies malicious intent regardless of the specific attack vector. The performance of JFrog, Protect AI, and ClamAV varies significantly across attack categories, highlighting the limitations of traditional scanning tools against sophisticated, framework-native manipulations.

To further verify MAD's capability against false positives, we also evaluate all methods using 40 carefully crafted benign models from the Advanced TensorAbuse Synthetic Dataset.

These models are specifically designed to invoke the same sensitive TensorFlow APIs flagged as attack vectors in the TensorAbuse approach. All such invocations serve legitimate purposes, including loading local dataset configurations, printing debug information and performing standard tensor persistence. Since these are officially supported TensorFlow APIs frequently used in real-world pipelines, this dataset provides a realistic benchmark for evaluating whether a detection tool can distinguish between legitimate and malicious API usage.

The precision results on the Advanced TensorAbuse Synthetic Dataset in Table III clearly reveal the drawbacks of static detection mechanisms. TensorAbuse misclassifies all 40 benign models as malicious, resulting in a 100% false positive rate. It relies solely on static scanning against a predefined blacklist of high-risk APIs and lacks the ability to understand API semantics and invocation context. Similarly, JFrog and Protect AI also produce numerous misclassifications. This indicates that signature-based detection methods fail when legitimate operations overlap with known attack patterns.

In contrast, MAD successfully identifies all 40 models as benign. By dynamically observing runtime execution, MAD's first stage recognizes that the system calls generated by these APIs align with the established statistical baseline of normal framework operations. When a suspicious operation triggers cross-layer analysis, the LLM-based reasoning engine successfully interprets high-level Python semantics to validate benign intent. This demonstrates that MAD bridges the semantic gap, providing accurate defense without impeding legitimate workflows.

Overall, experimental results on both datasets prove that MAD is a robust defense solution with strong generalization capability. It can accurately detect malicious AI models exploiting common deserialization vulnerabilities or sophisticated and stealthy computational graph tampering. Furthermore, equipped with cross-layer semantic reasoning, MAD

TABLE V: Runtime execution overhead and detection latency of MAD evaluated on standard foundation models.

Model	Execution Time (s)			Detection Latency (s)
	Vanilla	w/ MAD	Overhead	
BERT	11.4857	12.1767	6.02 %	13.5014
YamNet	16.3302	17.1434	4.98 %	10.3641
EfficientNet	24.4290	25.8889	5.97 %	6.5911
VGG16	25.0444	26.4766	5.46 %	8.7171
Average	19.3223	20.4214	5.69 %	9.7934

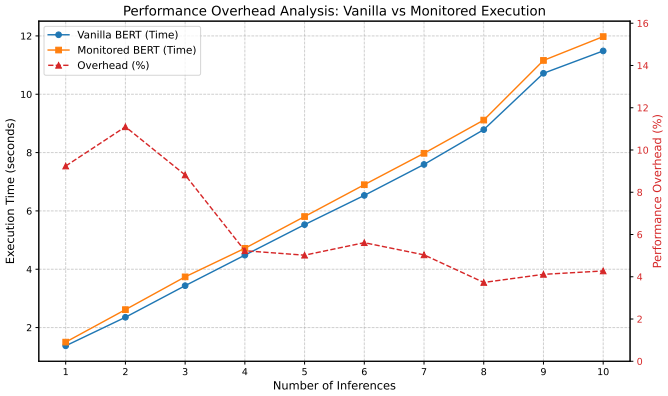

Fig. 3: Performance overhead analysis showing the execution time of Vanilla vs. Monitored BERT and the relative overhead percentage across varying numbers of inferences.

effectively differentiates legitimate usage of sensitive framework APIs from malicious attacks. It achieves high precision on both datasets without interfering with developers' regular workflows.

C. RQ2: Efficiency and Deployability

To evaluate the practical deployability of MAD, we investigate its impact on the host system's performance and its responsiveness to threats. It is important to note that we do not compare MAD with the baseline methods in this evaluation. The baseline methods are strictly static analysis tools designed to scan model artifacts pre-deployment, whereas MAD is a dynamic, runtime defense mechanism. Comparing the one-time scanning duration of static tools against the continuous runtime overhead of a dynamic monitor is fundamentally inequitable. Therefore, our evaluation focuses on the absolute performance impact of MAD on standard model execution.

We assess efficiency using two primary metrics:

- *Performance Overhead:* The degradation in model loading time and inference throughput (Samples/Second) caused by deploying MAD.
- *Detection Latency:* The time elapsed from the exact moment a malicious operation is initiated by the model to the generation of the final alert (including Autoencoder screening and Large language model (llm) reasoning).

Performance Overhead Analysis. The performance impact of MAD on normal model execution is presented in Table V. The results demonstrate that MAD introduces marginal overhead to the target system. Specifically, the total execution time increases by an average of only 5.69%, with the overhead consistently remaining within a narrow 4.9% to 6.0% margin across fundamentally different model architectures.

This performance penalty is attributed to our architectural design. First, the application-layer dynamic instrumentation employs a highly lightweight boundary-hooking mechanism. Instead of deeply traversing and serializing complex, multi-gigabyte Tensor objects, it only captures high-level metadata. More importantly, the core analytical engines of MAD—the Autoencoder screening module and the llm reasoning module—run entirely asynchronously on the CPU. Because they do not compete for GPU memory or compute cycles, the intensive matrix multiplications and data parallelism required for AI inference remain completely unhindered.

To further demonstrate that our detection system minimally impacts inference efficiency, we conducted an in-depth analysis of the relationship between the number of inference iterations and the relative latency overhead. Figure 3 illustrates the execution time of the BERT model over 1 to 10 inference steps, comparing the vanilla execution with the MAD-monitored execution. As observed, the relative overhead percentage is highest during the initial executions but quickly drops and stabilizes at a lower baseline as the number of inferences increases.

This phenomenon aligns well with the operating characteristics of deep learning frameworks. The model loading and initialization phases generate a large number of intensive system calls, such as memory mapping operations and heavy file I/O for loading model weights, which incur temporary CPU-bound monitoring overhead. However, once the model enters the continuous inference phase, the intensive computations are primarily offloaded to hardware accelerators. These computations generate very few new system calls and do not occupy main CPU resources. Therefore, the overhead introduced by dynamic instrumentation and system call monitoring gradually diminishes.

Detection Latency Analysis. As shown in Table V, MAD achieves an average detection latency of 9.79 seconds. This latency encompasses the full analytical pipeline: capturing the low-level system calls, completing the Autoencoder forward pass, extracting the corresponding Python contextual logs, and generating the final llm intent classification. It is crucial to note that the llm intent classification is only invoked on a small fraction of operations marked as abnormal by the Autoencoder, so its cost does not scale with total runtime but solely with the number of suspicious events.

While dynamic reasoning inherently introduces some delay compared to simple signature matching, this two-stage design keeps the overall overhead well within the acceptable budget for offline model vetting pipelines and pre-deployment security checks. Our experiments suggest that MAD can be seamlessly integrated as the final security gate in the model release pipeline with acceptable latency during pre-deployment security testing.

TABLE VI: Ablation study of MAD on detection performance, efficiency, and llm token overhead. Latency represents the average detection time per model, and Token Cost represents the average number of tokens consumed per model.

Method	Precision	Recall	F1	Latency (s)	Token Cost
MAD-S	90.91%	44.30%	59.57%	30.3126	867075
MAD-L	85.25%	**98.99%**	91.61%	**8.9778**	**50005**
MAD	**100.00%**	**98.99%**	**99.49%**	9.7934	57207.1

D. RQ3: Component Ablation and Sensitivity

In this section, we evaluate the contribution of each core component of MAD through ablation studies. To demonstrate the necessity of our two-stage cross-layer architecture, we evaluate two variants of MAD:

- *MAD–S*: which removes the unsupervised Autoencoder screening stage, directly feeding all raw system call windows and application-level instrumentation logs to the llm for anomaly detection;
- *MAD–L*: which removes the application-level dynamic instrumentation, relying solely on the anomalous system call operations filtered by the Autoencoder for the llm to make its final judgment without high-level Python semantic context;

We evaluate the performance of these two variants and compare them with the full MAD model. The evaluation metrics cover detection accuracy, including precision, recall, and F1-score, as well as system efficiency involving detection latency and LLM token cost. Table VI details the comprehensive metrics.

Overall, the results show that both variants of MAD exhibit significantly degraded performance or unacceptable system overhead compared to the full model, proving that both stages are indispensable.

Specifically, MAD-S suffers from a catastrophic decrease in detection efficiency and a massive surge in llm token consumption. Because it bypasses the Autoencoder screening, the llm is overwhelmed by the massive volume of benign, repetitive system events generated during normal model loading and inference. This increases the average detection latency to an impractical level (20.5 seconds) and incurs extreme API costs (809868 tokens).In addition, it slightly reduces precision, as the llm is more prone to hallucination when processing excessive background noise. This demonstrates that the statistical screening stage is crucial for filtering out normal activity and maintaining the practical deployability of the system.

On the other hand, MAD-L performs the worst in terms of detection accuracy, particularly suffering a severe drop in precision (14.75% lower than the full model). By removing the application-level instrumentation logs, the llm is forced to judge intent based entirely on low-level system calls. Without the high-level semantic context such as function arguments to explain why a system call was executed, benign operations are frequently misclassified as malicious network exfiltration. This confirms our prior observation that a severe semantic gap exists at the OS level, and dynamic cross-layer context alignment is the most important component for eliminating false positives and accurately discerning the true intent of AI models.

VI. Related Work

Adversarial Attacks and Defenses on AI Models. Existing security research on AI models has predominantly focused on threats that manipulate the model's inference outcomes or training integrity. These include adversarial attacks [39]–[41], where imperceptible perturbations cause misclassification; data poisoning [42]–[44], which corrupts the training data to degrade performance; and backdoor attacks [45]–[47], which inject hidden triggers into models. While extensive defense mechanisms like adversarial training and robust distillation have been proposed for these threats, they fundamentally address the statistical vulnerabilities of machine learning algorithms. In contrast, our work focuses on the comparatively under-investigated domain of *AI supply chain security*, where attackers exploit the model artifacts not to manipulate AI predictions, but to execute arbitrary, system-level malicious code on the victim's host machine.

Malicious Code Poisoning in the AI Supply Chain. With the rapid growth of open-source model hubs, the AI supply chain has become a prime target for attackers. Attackers embed malicious code into model artifacts to spread malware for profits. Early studies primarily focused on the vulnerabilities inherent in insecure serialization formats. Research has extensively documented how formats like Python's `pickle` can be exploited to execute arbitrary code during the deserialization process [26]. To mitigate these threats, several static analysis and scanning tools have been developed. For instance, Fickling [7] decompiles `pickle` bytecodes to identify suspicious patterns, while Picklescanning [8] and ModelScan [6] integrate with antivirus engines like ClamAV [33] to detect known malware signatures within model files. However, these defense mechanisms are predominantly static and rely heavily on predefined signatures or rules. Consequently, they are susceptible to evasion techniques and often generate high false positive rates when legitimate model operations resemble malicious patterns.

Framework-Native Attacks and API Abuse. Recent advancements in supply chain attacks have moved beyond simple serialization exploits towards more sophisticated, framework-native methodologies. These attacks leverage the legitimate, built-in APIs of AI frameworks to execute malicious behaviors, making them exceptionally stealthy. Zhu et al. [11] introduced TensorAbuse, a novel attack paradigm that exploits the hidden capabilities of TensorFlow APIs to construct persistent and stealthy attacks directly embedded within the model's computational graph. Because these attacks utilize officially supported framework operations rather than external system calls or standard malicious payloads, they effectively bypass traditional static security scanners. While the authors proposed an LLM-based tool to statically classify API capabilities and detect potentially exploitable APIs, this approach still operates

at a static level and struggles with the semantic ambiguity of API usage, leading to potential inaccuracies in distinguishing between benign and malicious intents. In contrast, our proposed method, MAD, introduces a dynamic, cross-layer approach that monitors actual runtime behavior and leverages high-level execution context to accurately identify malicious intent, addressing the limitations of existing static analysis and signature-based defenses.

System Call-Based Detection. System call sequence analysis serves as a core technique for traditional malware detection, yet relevant research remains scarce in the field of AI model detection []. Recent studies have abandoned early heuristic detection schemes and adopted advanced deep learning techniques to capture complex API behaviors. [48], [49] For instance, API2Vec++ [50] adopts temporal and attribute graphs embedding methods to characterize contextual behaviors inside and between processes, effectively solving the problem of interleaved API calls. Similarly, API-MalDetect [51] employs CNNs and BiGRUs to extract deep semantic features from long API sequences. CruParamer [52] incorporates parameter sensitivity into API embeddings to enhance detection accuracy. Inspired by the proven effectiveness of system call analysis in traditional security domains, the proposed MAD framework adapts this low-level monitoring strategy to AI scenarios. It establishes a robust framework-agnostic detection foundation and effectively defends against evasion attacks caused by application-layer obfuscation.

Unsupervised Anomaly Detection. Unsupervised learning is highly effective for identifying zero-day anomalies without relying on labeled attack data [53]. In the security domain, reconstruction-based models like AutoEncoders are widely utilized to establish benign behavioral baselines [54], [55]. For example, ContraMTD [56] successfully applied ensemble learning with AutoEncoders to detect anomalous network traffic. Similar reconstruction and clustering techniques have been deployed for hardware trojan detection [57] and computer vision anomalies [58]. Leveraging these unsupervised principles, MAD employs a sequence-to-sequence AutoEncoder to statistically model the highly regular, benign execution templates of AI frameworks. This enables our system to flag structural deviations caused by stealthy model supply-chain attacks without requiring any prior knowledge of specific malware signatures.

VII. Discussion

Scope of detection. MAD is designed to detect malicious behaviors that a model artifact executes against the host system, such as data exfiltration. Its detection pipeline relies on observing the actual runtime interactions between the model process and the operating system. MAD could not address attacks that target the model itself without affecting the host system. In particular, model-level threats such as neural backdoor injection fall outside our detection scope. These attacks embed malicious functionality into the model's learned parameters, and at inference time the compromised model performs only normal tensor computations through standard framework operations. So these attacks generate no anomalous system calls or suspicious API invocations that MAD relies upon as its primary detection signal.

Consequently, these attacks are invisible to MAD's cross-layer monitoring pipeline. Such attacks compromise the model's prediction integrity rather than its runtime behavior, and their malicious effects manifest as incorrect or biased outputs rather than as anomalous system calls or API invocations. Defending against such threats requires complementary techniques such as model provenance verification, weight integrity checking, and robust training, which are orthogonal to and compatible with MAD's host-level defense.

VIII. Conclusion

Malicious model artifacts present a growing threat to the AI supply chain: attackers can embed harmful payloads within serialized models that execute during loading or inference, evading traditional static scanners by using legitimate framework APIs. In this paper, we propose MAD, a hybrid runtime defense system that combines non-bypassable kernel-level system-call monitoring with LLM-assisted Python semantic instrumentation to detect malicious behaviors in model artifacts. Instead of relying solely on static signature matching, MAD captures the actual runtime execution evidence through a two-stage architecture: an unsupervised Autoencoder prefilter that surfaces anomalous low-level operations, and a cross-layer LLM reasoning engine that aligns these operations with their high-level semantic context to determine malicious intent. We evaluate MAD on the MalHug Dataset and advanced synthetic attacks. The results show that MAD achieves near-perfect detection, while introducing an average runtime overhead of only 5.69%, demonstrating that MAD provides a practical, deployable defense for the AI model supply chain.

References

[1] H. Face, "Hugging face – the ai community building the future." https://huggingface.co/, 2026, https://huggingface.co/.

[2] P. Girnus, F. Tucci, D. Patel, S. Dulude, A. Verma, and J. R. Navato, "Your ai gateway was a backdoor: Inside the litellm supply chain compromise," https://www.trendmicro.com/en_us/research/26/c/inside-litellm-supply-chain-compromise.html, 2026, access:2026-05-18. [Online]. Available: https://www.trendmicro.com/en_us/research/26/c/inside-litellm-supply-chain-compromise.html

[3] P. P. Index, "Incident report: Litellm/telnyx supply-chain attacks, with guidance," https://blog.pypi.org/posts/2026-04-02-incident-report-litellm-telnyx-supply-chain-attack/, 2026, access:2026-05-18. [Online]. Available: https://blog.pypi.org/posts/2026-04-02-incident-report-litellm-telnyx-supply-chain-attack/

[4] unit42, "Weaponizing the protectors: Teampcp's multi-stage supply chain attack on security infrastructure," https://origin-unit42.paloaltonetworks.com/teampcp-supply-chain-attacks/, 2026, access:2026-05-18. [Online]. Available: https://origin-unit42.paloaltonetworks.com/teampcp-supply-chain-attacks/

[5] J. Zhao, S. Wang, Y. Zhao, X. Hou, K. Wang, P. Gao, Y. Zhang, C. Wei, and H. Wang, "Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2087–2098. [Online]. Available: https://doi.org/10.1145/3691620.3695271

[6] Protect AI, "Modelscan: Protection against model serialization attacks," 2021, access:2026-05-18. [Online]. Available: https://github.com/protectai/modelscan

[7] Trail of Bits, "fickling," 2021, access:2026-05-18. [Online]. Available: https://github.com/trailofbits/fickling

[8] Hugging Face, "Pickle scanning," 2024, access:2026-05-18. [Online]. Available: https://huggingface.co/docs/hub/security-pickle#pickle-scanning

[9] A. Virkud, M. A. Inam, A. Riddle, J. Liu, G. Wang, and A. Bates, "How does endpoint detection use the MITRE ATT&CK framework?" in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3891–3908. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/virkud

[10] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, "MAGIC: Detecting advanced persistent threats via masked graph representation learning," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 5197–5214. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/jia-zian

[11] R. Zhu, G. Chen, W. Shen, X. Xie, and R. Chang, " My Model is Malware to You: Transforming AI Models into Malware by Abusing TensorFlow APIs ," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 449–466. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00012

[12] L. Gambacorta and V. Shreeti, "The ai supply chain," *Review of Network Economics*, vol. 24, no. 4, pp. 205–223, 2025. [Online]. Available: https://doi.org/10.1515/rne-2025-0063

[13] Hugging Face, "Models - hugging face," https://huggingface.co/models, 2026.

[14] W. Jiang, N. Synovic, M. Hyatt, T. R. Schorlemmer, R. Sethi, Y.-H. Lu, G. K. Thiruvathukal, and J. C. Davis, "An empirical study of pre-trained model reuse in the hugging face deep learning model registry," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2463–2475.

[15] M. Taraghi, G. Dorcelus, A. Foundjem, F. Tambon, and F. Khomh, "Deep learning model reuse in the huggingface community: Challenges, benefit and trends," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2024, pp. 512–523.

[16] M. L. Siddiq, T. H. Romel, N. Sekerak, B. Casey, and J. C. S. Santos, "An empirical study on remote code execution in machine learning model hosting ecosystems," 2026. [Online]. Available: https://arxiv.org/abs/2601.14163

[17] HiddenLayer, "Ai threat landscape 2026," https://www.hiddenlayer.com/report-and-guide/threatreport2026, 2026, access:2026-05-18. [Online]. Available: https://www.hiddenlayer.com/report-and-guide/threatreport2026

[18] Z. Wang, C. Liu, X. Cui, J. Yin, and X. Wang, "Evilmodel 2.0: Bringing neural network models into malware attacks," *Computers & Security*, vol. 120, p. 102807, 2022. [Online]. Available: http://dx.doi.org/10.1016/j.cose.2022.102807

[19] A. Kathikar, A. Nair, B. Lazarine, A. Sachdeva, and S. Samtani, "Assessing the vulnerabilities of the open-source artificial intelligence (ai) landscape: A large-scale analysis of the hugging face platform," in *2023 IEEE International Conference on Intelligence and Security Informatics (ISI)*, 2023, pp. 1–6.

[20] W. Jiang, N. Synovic, R. Sethi, A. Indarapu, M. Hyatt, T. R. Schorlemmer, G. K. Thiruvathukal, and J. C. Davis, "An empirical study of artifacts and security risks in the pre-trained model supply chain," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, ser. SCORED'22. New York, NY, USA: Association for Computing Machinery, 2022, p. 105–114. [Online]. Available: https://doi.org/10.1145/3560835.3564547

[21] N.-J. Huang, C.-J. Huang, and S.-K. Huang, "Pain pickle: Bypassing python restricted unpickler for automatic exploit generation," in *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, 2022, pp. 1079–1090.

[22] T. A. Nguyen, T. H. M. Le, and M. A. Babar, "Securing the ai supply chain: What can we learn from developer-reported security issues and solutions of ai projects?" 2026. [Online]. Available: https://arxiv.org/abs/2512.23385

[23] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[24] N. Koutroumpouchos, G. Lavdanis, E. Veroni, C. Ntantogian, and C. Xenakis, "Objectmap: detecting insecure object deserialization," in *Proceedings of the 23rd Pan-Hellenic Conference on Informatics*, ser. PCI '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 67–72. [Online]. Available: https://doi.org/10.1145/3368640.3368680

[25] A. D. Kellas, N. Christou, W. Jiang, P. Li, L. Simon, Y. David, V. P. Kemerlis, J. C. Davis, and J. Yang, "Pickleball: Secure deserialization of pickle-based machine learning models," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 3341–3355. [Online]. Available: https://doi.org/10.1145/3719027.3765037

[26] B. Casey, J. C. S. Santos, and M. Mirakhorli, "A large-scale exploit instrumentation study of ai/ml supply chain attacks in hugging face models," 2024. [Online]. Available: https://arxiv.org/abs/2410.04490

[27] Python Software Foundation, "pickle — python object serialization," https://docs.python.org/3/library/pickle.html, 2026, accessed: 2026-05-25.

[28] T. Liu, G. Meng, P. Zhou, Z. Deng, S. Yao, and K. Chen, "The art of hide and seek: Making pickle-based model supply chain poisoning stealthy again," 2025. [Online]. Available: https://arxiv.org/abs/2508.19774

[29] The HDF Group, "High-performance data management and storage suite," https://www.hdfgroup.org/HDF5/, accessed: 2026-05-25.

[30] safetensors, "Safetensors," https://github.com/huggingface/safetensors, accessed: 2026-05-25.

[31] onnx, "ONNX," https://github.com/onnx/onnx, 2020, accessed: 2026-05-25.

[32] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: A system for Large-Scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. [Online]. Available: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

[33] Cisco Systems, "ClamAVNet," https://www.clamav.net/, access:2026-05-25. [Online]. Available: https://www.clamav.net/

[34] H. Siadati, S. Jafarikhah, E. Sahin, T. Hernandez, E. Tripp, D. Khryashchev, and A. Kharraz, "Devphish: Exploring social engineering in software supply chain attacks on developers," in *2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, 2024, pp. 517–523.

[35] S. Yun, Y. Zhang, Z. Shi, L. Wang, Y. Wang, and Y. Bai, "Large-scale empirical study of secret keys leakage in hugging face spaces," 05 2026.

[36] M. Dimjašević, S. Atzeni, I. Ugrina, and Z. Rakamaric, "Evaluation of android malware detection based on system calls," in *Proceedings of the 2016 ACM on International Workshop on Security And Privacy Analytics*, ser. IWSPA '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 1–8. [Online]. Available: https://doi.org/10.1145/2875475.2875487

[37] S. Shakya and M. Dave, "Analysis, detection, and classification of android malware using system calls," 2022. [Online]. Available: https://arxiv.org/abs/2208.06130

[38] David Cohen, "Data scientists targeted by malicious hugging face ml models with silent backdoor," https://jfrog.com/blog/data-scientists-targeted-by-malicious-hugging-face-ml-models-with-silent-backdoor/, 2024.

[39] J. C. Costa, T. Roxo, H. Proença, and P. R. M. Inácio, "How deep learning sees the world: A survey on adversarial attacks & defenses," *IEEE Access*, vol. 12, pp. 61 113–61 136, 2024.

[40] Y. Ren, Z. Zhao, C. Lin, B. Yang, L. Zhou, Z. Liu, and C. Shen, "Improving adversarial transferability on vision transformers via forward propagation refinement," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 25 071–25 080.

[41] P. Xie, Y. Bie, J. Mao, Y. Song, Y. Wang, H. Chen, and K. Chen, "Chain of attack: On the robustness of vision-language models against transfer-based adversarial attacks," in *Proceedings of the IEEE/CVF Conference*

on Computer Vision and Pattern Recognition (CVPR), June 2025, pp. 14 679–14 689.

[42] D. Bowen, B. Murphy, W. Cai, D. Khachaturov, A. Gleave, and K. Pelrine, "Scaling trends for data poisoning in llms," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 26, p. 27206–27214, Apr. 2025. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/34929

[43] Y. Zeng, M. Pan, H. Jahagirdar, M. Jin, L. Lyu, and R. Jia, "Meta-Sift: How to sift out a clean subset in the presence of data poisoning?" in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 1667–1684. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/zeng

[44] A. E. Cinà, K. Grosse, A. Demontis, B. Biggio, F. Roli, and M. Pelillo, "Machine learning security against data poisoning: Are we there yet?" *Computer*, vol. 57, no. 3, pp. 26–34, 2024.

[45] Y. Zeng, M. Pan, H. A. Just, L. Lyu, M. Qiu, and R. Jia, "Narcissus: A practical clean-label backdoor attack with limited information," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 771–785. [Online]. Available: https://doi.org/10.1145/3576915.3616617

[46] H. Wang, Z. Xiang, D. J. Miller, and G. Kesidis, "Mm-bd: Post-training detection of backdoor attacks with arbitrary backdoor pattern types using a maximum margin statistic," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 1994–2012.

[47] G. Shen, S. Cheng, Z. Zhang, G. Tao, K. Zhang, H. Guo, L. Yan, X. Jin, S. An, S. Ma, and X. Zhang, "Bait: Large language model backdoor scanning by inverting attack target," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 1676–1694.

[48] D. Gibert, C. Mateu, and J. Planes, "The rise of machine learning for detection and classification of malware: Research developments, trends and challenges," *Journal of Network and Computer Applications*, 03 2020.

[49] B. Kolosnjaji, A. Zarras, G. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in *AI 2016: Advances in Artificial Intelligence*, B. H. Kang and Q. Bai, Eds. Cham: Springer International Publishing, 2016, pp. 137–149.

[50] L. Cui, J. Yin, J. Cui, Y. Ji, P. Liu, Z. Hao, and X. Yun, "Api2vec++: Boosting api sequence representation for malware detection and classification," *IEEE Transactions on Software Engineering*, vol. 50, no. 8, pp. 2142–2162, 2024.

[51] P. Maniriho, A. N. Mahmood, and M. J. M. Chowdhury, "Apimaldetect: Automated malware detection framework for windows based on api calls and deep learning techniques," *J. Netw. Comput. Appl.*, vol. 218, no. C, Sep. 2023. [Online]. Available: https://doi.org/10.1016/j.jnca.2023.103704

[52] X. Chen, Z. Hao, L. Li, L. Cui, Y. Zhu, Z. Ding, and Y. Liu, "Cruparamer: Learning on parameter-augmented api sequences for malware detection," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 788–803, 2022.

[53] R. Bouman, Z. Bukhsh, and T. Heskes, "Unsupervised anomaly detection algorithms on real-world data: How many do we need?" *Journal of Machine Learning Research*, vol. 25, no. 105, pp. 1–34, 2024. [Online]. Available: http://jmlr.org/papers/v25/23-0570.html

[54] K. Berahmand, F. Daneshfar, E. Salehi, Y. Li, and Y. Xu, "Autoencoders and their applications in machine learning: a survey," *Artificial Intelligence Review*, vol. 57, 02 2024.

[55] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018. [Online]. Available: https://arxiv.org/abs/1802.09089

[56] X. Han, S. Cui, J. Qin, S. Liu, B. Jiang, C. Dong, Z. Lu, and B. Liu, "Contramtd: An unsupervised malicious network traffic detection method based on contrastive learning," in *Proceedings of the ACM Web Conference 2024*, ser. WWW '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1680–1689. [Online]. Available: https://doi.org/10.1145/3589334.3645479

[57] T. Gourousis, Z. Zhang, M. Yan, M. Zhang, A. Mittal, A. Shrivastava, F. Restuccia, Y. Fei, and M. Onabajo, "Identification of stealthy hardware trojans through on-chip temperature sensing and an autoencoder-based machine learning algorithm," in *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*, 2023, pp. 30–34.

[58] X. Zhang, N. Li, J. Li, T. Dai, Y. Jiang, and S.-T. Xia, "Unsupervised surface anomaly detection with diffusion probabilistic model," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 6782–6791.